

SQL 강의노트 복습

- (3주차) 데이터 검색과 그룹핑
 - DB 및 테이블 확인하기
 - SHOW DATABASES
 - USE
 - SHOW TABLES
 - DESCRIBE / DESC
 - DB / 테이블 생성
 - DROP DATABASE IF EXISTS / CREATE
 - CREATE TABLE
 - INSERT INTO table VALUES()
 - 데이터 추출 기초
 - SELECT
 - WHERE
 - AND, OR, BETWEEN, IN(), LIKE
 - WHERE 절에 Subquery로 조건 필터링하기
 - ANY, ALL
 - ORDER BY
 - DISTINCT
 - LIMIT
 - 데이터 집계
 - GROUP BY
 - 집계함수 (MIN, MAX, AVG, SUM, COUNT, STDDEV, VAR_SAMP)
 - HAVING
 - WITH ROLLUP
- (4주차) 4주차 데이터 삽입, 수정, 삭제와 WITH 절
 - 데이터 삽입, 수정, 삭제
 - INSERT
 - AUTO_INCREMENT
 - UPDATE
 - DELETE
 - DELETE, DROP, TRUNCATE 비교
 - 윈도우 함수
 - ROW_NUMBER
 - DENSE_RANK
 - RANK
 - NTILE
 - LEAD
 - FIRST_VALUE
 - CUME_DIST
 - 피벗
 - CTE (Common Table Expression)
 - WITH
- (5주차) 데이터 형식과 내장 함수

- 데이터 형식
 - 숫자데이터
 - 문자 데이터
 - 날짜와 시간 데이터
- 데이터 형식 변환 함수
 - CAST, CONVERT
 - FORMAT_DATETIME
 - 암시적인 형 변환
- 변수의 선언과 활용
 - SET
 - PREPARE & EXECUTE
- (내장 함수) 제어 흐름 함수
 - IF
 - IFNULL
 - NULLIF
 - CASE ... WHEN ... ELSE ... END
- (내장 함수) 문자열 함수
 - ASCII & CHAR
 - BIT_LENGTH, CHAR_LENGTH, LENGTH
 - CONCAT, CONCAT_WS
 - FORMAT
 - INSERT
 - LOWER와 UPPER
 - LTRIM, RTRIM
 - REPLACE
 - REVERSE
 - SUBSTRING
- (내장 함수) 수학 함수
 - POW(숫자1, 숫자2), SQRT(숫자)
 - RAND
- (내장 함수) 날짜/시간 함수
 - ADDDATE(날짜, 차이), SUBDATE(날짜, 차이)
 - ADDTIME(날짜/시간, 시간), SUBTIME(날짜/시간, 시간)
 - YEAR(날짜), MONTH(날짜), DAY(날짜), HOUR(시간), MINUTE(시간), SECOND(시간), MICROSECOND(시간)
 - DATE(), TIME()
 - DATEDIFF(날짜1, 날짜2), TIMEDIFF(날짜1/시간1, 날짜1/시간2)
 - DAYOFWEEK(날짜), MONTHNAME(날짜), DAYOFYEAR(날짜)
 - LAST_DAY(날짜)
 - MAKEDATE(연도, 정수)
 - TIME_TO_SEC(시간)
 - CURDATE(), CURTIME(), NOW(), SYSDATE()
- (내장 함수) 시스템 정보 함수
 - USER(), DATABASE()
 - FOUND_ROWS()
 - ROW_COUNT()

- (7주차) 조인과 SQL 프로그래밍
 - JOIN 기본 개념
 - JOIN 기본 개념
 - INNER JOIN
 - OUTER JOIN
 - CROSS JOIN
 - SELF JOIN
 - UNION
 - IN, NOT IN
 - SQL 프로그래밍
 - IF ... ELSE ... END IF
 - CASE
 - WHILE
 - PREPARE, EXECUTE (동적 SQL문)

(3주차) 데이터 검색과 그룹핑

추출을 위한 SQL 쿼리문은 다음과 같은 형식으로 요약된다.

```
SELECT select_expr
  [FROM table_references]
  [WHERE where_condition]
  [GROUP BY (col_name [expr | position])]
  [HAVING where_condition]
  [ORDER BY (col_name |expr | position)]
```

각각의 라인에 해당하는 문법의 상세 설명을 살펴보도록 하자.

DB 및 테이블 확인하기

SHOW DATABASES

- 현재 서버에 어떤 데이터베이스가 있는지 조회

```
SHOW DATABASES;
```

USE

- 현재 사용하는 데이터베이스를 지정하거나 변경하는 구문 형식

```
USE 데이터베이스이름;
```

SHOW TABLES

- 현재 사용하고자 하는 데이터베이스에 어떤 테이블들이 있는지 확인

```
USE 데이터베이스이름;
SHOW TABLES;
```

DESCRIBE / DESC

- 특정 테이블 열에 무엇이 있는지 확인

```
DESCRIBE employees;  
DESC employees;
```

DB / 테이블 생성

DROP DATABASE IF EXISTS / CREATE

- 만약 데이터베이스가 이미 존재한다면 DROP
- `CREATE` 을 통해서 데이터베이스 생성

```
DROP DATABASE IF EXISTS cookDB; -- 만약 cookDB가 존재하면 우선 삭제한다.  
CREATE DATABASE cookDB;
```

CREATE TABLE

- 데이터베이스안에 테이블을 생성하고, 각각의 컬럼을 선언한다.

```
USE cookDB;  
CREATE TABLE userTBL -- 회원 테이블  
(  
  userID CHAR(8) NOT NULL PRIMARY KEY, -- 사용자 아이디(PK)  
  userName VARCHAR(10) NOT NULL, -- 이름  
  birthYear INT NOT NULL, -- 출생 연도  
  addr CHAR(2) NOT NULL, -- 지역(경기, 서울, 경남 식으로 2글자만 입력)  
  mobile1 CHAR(3), -- 휴대폰의 국번(011, 016, 017, 018, 019, 010 등)  
  mobile2 CHAR(8), -- 휴대폰의 나머지 번호(하이픈 제외)  
  height SMALLINT, -- 키  
  mDate DATE -- 회원 가입일  
);  
  
CREATE TABLE buyTBL -- 구매 테이블  
(  
  num INT AUTO_INCREMENT NOT NULL PRIMARY KEY, -- 순번(PK)  
  userID CHAR(8) NOT NULL, -- 아이디(FK)  
  prodName CHAR(6) NOT NULL, -- 물품 groupName CHAR(4), -- 분류  
  price INT NOT NULL, -- 단가  
  amount SMALLINT NOT NULL, -- 수량  
  FOREIGN KEY (userID) REFERENCES userTBL (userID)  
);
```

INSERT INTO table VALUES()

- 테이블에 데이터를 삽입

```
INSERT INTO userTBL VALUES ('YJS', '유재석', 1972, '서울', '010', '11111111', 178, '2008-8-8')  
INSERT INTO userTBL VALUES ('KHD', '강호동', 1970, '경북', '011', '22222222', 182, '2007-7-7')  
INSERT INTO userTBL VALUES ('KKJ', '김국진', 1965, '서울', '019', '33333333', 171, '2009-9-9')  
INSERT INTO userTBL VALUES ('KYM', '김용만', 1967, '서울', '010', '44444444', 177, '2015-5-5')  
INSERT INTO userTBL VALUES ('KJD', '김제동', 1974, '경남', NULL, NULL, 173, '2013-3-3');  
INSERT INTO userTBL VALUES ('NHS', '남희석', 1971, '충남', '016', '66666666', 180, '2017-4-4')
```

```
INSERT INTO userTBL VALUES ('SDY', '신동엽', 1971, '경기', NULL, NULL, 176, '2008-10-10');
INSERT INTO userTBL VALUES ('LHJ', '이휘재', 1972, '경기', '011', '88888888', 180, '2006-4-4');
INSERT INTO userTBL VALUES ('LKK', '이경규', 1960, '경남', '018', '99999999', 170, '2004-12-');
INSERT INTO userTBL VALUES ('PSH', '박수홍', 1970, '서울', '010', '00000000', 183, '2012-5-5');
```

데이터 추출 기초

SELECT

- 추출할 컬럼을 선택

```
SELECT
    열이름
FROM
    테이블이름
```

WHERE

- 추출할 Row에 대한 조건 설정

```
SELECT
    열이름
FROM
    테이블이름
WHERE
    조건식
```

AND, OR, BETWEEN, IN(), LIKE

- `WHERE` 절에 조건을 걸어줄 때 많이 사용되는 문법들

AND

```
SELECT
    userID,
    userName
FROM
    userTBL
WHERE
    birthYear >= 1970
    AND height >= 182;
```

OR

```
SELECT
    userID,
    userName
FROM
    userTBL
WHERE
```

```
birthYear >= 1970  
OR height >= 182;
```

BETWEEN

- 위와 아래 문법은 결과가 동일함

```
SELECT  
    userName,  
    height  
FROM  
    userTBL  
WHERE  
    height >= 180  
    AND height <= 182;  
  
SELECT  
    userName,  
    height  
FROM  
    userTBL  
WHERE  
    height BETWEEN 180 AND 182;
```

IN

- OR 조건을 한번에 복수개의 조건으로 한줄로 쓸 수 있음
- 아래 코드의 두 예시 결과는 같음

```
SELECT  
    userName,  
    addr  
FROM  
    userTBL  
WHERE  
    addr='경남' OR addr='충남' OR addr='경북';  
  
SELECT  
    userName,  
    addr  
FROM  
    userTBL  
WHERE  
    addr IN ('경남', '충남', '경북');
```

LIKE

- `%`, `_` 등의 정규식을 이용해서 이에 맞는 조건 출력 가능

-- 성이 김씨인 회원의 이름과 키 조회

```
SELECT
    userName,
    height
FROM
    userTBL
WHERE
    userName LIKE '김%';
```

-- 성을 한글자로 제한해서 이름이 경규인 사람 찾기

```
SELECT
    userName,
    height
FROM
    userTBL
WHERE
    userName LIKE '_경규';
```

WHERE 절에 Subquery로 조건 필터링하기

- 김용만의 키가 177이라고 하자. 이때 김용만보다 키가 크거나 같은 사람의 이름과 키 출력.
- 위와 아래 쿼리는 같은 결과를 출력한다.

```
SELECT
    userName,
    height
FROM
    userTBL
WHERE
    height > 177;
```

```
SELECT
    userName,
    height
FROM
    userTBL
WHERE
    height > (
        SELECT
            height
        FROM
            userTBL
        WHERE
            userName = '김용만'
    )
```

ANY, ALL

- 지역이 경기인 사람보다 키가 크거나 같은 사람 추출하고자 함.
- 이때 '경기'에 사는 사람은 여러명 존재하므로, ANY 혹은 ALL 을 사용해서 조건을 지정해 줄 수 있음
- ANY 를 사용할 경우, 어떤 하나라도 만족하는 경우 TRUE 반환

```
SELECT userName, height
FROM userTBL
WHERE height >= ANY (SELECT height FROM userTBL WHERE addr = '경기');
```

- ALL을 사용할 경우, 모두 만족해야지 TRUE 를 반환

```
SELECT userName, height
FROM userTBL
WHERE height >= ALL (SELECT height FROM userTBL WHERE addr = '경기');
```

주요 차이점

조건 비교	ANY	ALL
비교 방식	서브쿼리 결과 중 하나라도 조건을 만족하면 참	서브쿼리 결과 중 모두가 조건을 만족해야 참
연산 결과	더 느슨한 조건	더 엄격한 조건
예제	<code>>= ANY (180, 176, 182)</code> → 176 이상이면 참	<code>>= ALL (180, 176, 182)</code> → 182 이상이어야 참

요약:

- ANY : 하나라도 만족하면 OK!
- ALL : 전부 만족해야 OK!

=ANY (서브쿼리) 는 IN (서브쿼리)와 동일하다

```
SELECT userName, height FROM userTBL
WHERE height = ANY (SELECT height FROM userTBL WHERE addr = '경기');
```

```
SELECT userName, height FROM userTBL
WHERE height IN (SELECT height FROM userTBL WHERE addr = '경기');
```

ORDER BY

- 순서 정렬에 사용되는 문법
- Default는 오름차순(Ascending)이며, DESC를 넣을수도 있음

```
-- 오름차순
SELECT userName, mDate
FROM userTBL
ORDER BY mDate;

-- 내림차순
SELECT userName, mDate
FROM userTBL
ORDER BY mDate DESC;

-- 두 개 이상의 정렬 기준 사용
SELECT userName, height
```

```
FROM userTBL
ORDER BY height DESC, userName ASC;
```

DISTINCT

- unique한 값만 출력하고 싶은 경우 사용

```
SELECT DISTINCT addr
FROM userTBL;
```

LIMIT

- 출력하고자 하는 수를 제한할 때 사용

```
-- 오래된 입사일 기준 상위 5명의 사원번호(emp_no) 조회
SELECT emp_no, hire_date
FROM employees
ORDER BY hire_date ASC
LIMIT 5;

SELECT emp_no, hire_date
FROM employees
ORDER BY hire_date ASC
LIMIT 0, 5; -- LIMIT 5와 동일
```

(참고) CREATE TABLE와 서브쿼리를 이용해서도 테이블 생성 가능

- CREATE TABLE 새로운테이블 (SELECT 복사할열 FROM 기존테이블) 형태로 새로운 테이블 생성 가능
- 단, 기본키, 외래키 등의 제약 조건은 복사되지 않음

```
USE cookDB;
CREATE TABLE buyTBL2 (SELECT * FROM buyTBL);
```

데이터 집계

GROUP BY

- 기본적으로 집계를 하기 위한 기준을 설정할 때 사용됨
- 기본 문법 및 순서

```
SELECT select_expr
[FROM table_references]
[WHERE where_condition]
[GROUP BY {col_name | expr | position}]
[HAVING where_condition]
[ORDER BY {col_name | expr | position}]
```

예제

- 같은 ID(userID) 별로 구매 개수 합산하기

```
SELECT userID AS '사용자 아이디', SUM(amount) AS '총 구매 개수'
FROM buyTBL
GROUP BY userID;
```

- 같은 ID별로 구매액 총합 합산하기

```
SELECT userID AS '사용자 아이디', SUM(price * amount) AS '총 구매액'
FROM buyTBL
GROUP BY userID;
```

집계함수 (MIN, MAX, AVG, SUM, COUNT, STDDEV, VAR_SAMP)

- GROUP BY는 일반적으로 집계 함수와 함께 사용된다
- 자주 사용하는 집계 함수 모음은 다음과 같다.

함수	설명
AVG()	평균 값
SUM()	합계
COUNT()	행(row)의 개수
COUNT(DISTINCT)	중복을 제거한 고유 행(row)의 개수
MAX()	최대 값
MIN()	최소 값
STDDEV()	표준편차
VAR_SAMP()	분산(표본 기준)

예제 1

```
-- 평균 구하기
SELECT AVG(amount) AS '평균 구매 개수'
FROM buyTBL;

-- 최대값과 최소값 구하기
SELECT MAX(height) AS '최대 키', MIN(height) AS '최소 키'
FROM userTBL;
```

예시 2

- 앞서 살펴본 GROUP BY를 이용하는 경우에도 동일하게 다양한 집계 함수를 적용할 수 있다.

```
-- 회원별로 한 번 구매할 때마다 평균적으로 몇 개를 구매했는지 조회 (GROUP BY 절 사용)
SELECT userID, AVG(amount) AS '평균 구매 개수'
FROM buyTBL
GROUP BY userID;

-- 서버쿼리와 조합하여 가장 큰 키와 작은 키를 가진 회원 조회
SELECT userName, height
```

```

FROM userTBL
WHERE height = (SELECT MAX(height) FROM userTBL)
    OR height = (SELECT MIN(height) FROM userTBL);

-- 휴대폰이 있는 회원의 수만 조회
SELECT COUNT(mobile1) AS '휴대폰이 있는 사용자'
FROM userTBL;

```

HAVING

- GROUP BY를 통해서 집계된 결과에 대해서, 2차적으로 조건을 걸어주고 싶은 경우에 사용
- WHERE 절과 HAVING 절은 잘 구분하자

예시 1

```

-- 총 구매액이 적은 회원 순으로 정렬 (ORDER BY 절 사용)
SELECT userID AS '사용자', SUM(price * amount) AS '총구매액'
FROM buyTBL
GROUP BY userID
HAVING SUM(price * amount) > 1000
ORDER BY SUM(price * amount);

```

- GROUP BY 를 통해 사용자별로 데이터를 그룹화하고 총 구매액을 계산
- HAVING 절을 사용하여 총 구매액이 1000 이상인 사용자만 필터링
- ORDER BY 절을 사용하여 총 구매액을 오름차순으로 정렬

WITH ROLLUP

- GROUP BY를 기준으로 집계할 때, 그룹별 합계 및 총합을 계산한다.

```

-- 소합계와 총합만 필요하다면 num 제외
SELECT groupName, SUM(price * amount) AS '비용'
FROM buyTBL
GROUP BY groupName WITH ROLLUP;

```

- 각 groupName 에 대한 합계와 전체 총합이 출력
- 소합계: 각 groupName 별 합계.
- 총합계: 모든 데이터를 포함한 전체 합계가 마지막 행에 표시

(4주차) 4주차 데이터 삽입, 수정, 삭제와 WITH 절

데이터 삽입, 수정, 삭제

실질적으로 데이터 분석하는 입장에서 데이터를 삽입/수정/삭제 하는 것은 극히 드물지만, 일단 어떤 것이 있는지 대충 알아두도록 하자.

INSERT

- 테이블에 데이터를 삽입
- 쿼리문 형태는 일반적으로 아래와 같음

```
INSERT INTO 테이블이름 (열1, 열2, ...)
VALUES (값1, 값2, ...);
```

예시 1) 단일 Row 삽입

```
USE cookDB;
CREATE TABLE testTBL1 (
    id INT,
    userName CHAR(3),
    age INT
);

-- 열 이름은 생략 가능, 단 삽입하고자 하는 데이터와 열 순서는 맞춰줘야함
INSERT INTO testTBL1 VALUES (1, '뽀로로', 16);

-- id와 이름만 입력하고 나이는 입력하고 싶지 않은 경우
INSERT INTO testTBL1 (id, userName) VALUES (2, '크롱');
```

예시 2) 여러 데이터를 한꺼번에 삽입하기

```
INSERT INTO testTBL3 VALUES
    (NULL, '토이', 17),
    (NULL, '스토리', 18),
    (NULL, '무비', 19);

SELECT * FROM testTBL3;
```

예시 3) 대량 데이터 삽입 형식

```
INSERT INTO 테이블이름 (열1, 열2, ...)
SELECT 열1, 열2, ...
FROM 원본테이블;
```

예시 4) 다른 테이블의 데이터를 가져와 삽입하기

```
USE cookDB;

-- 대상 테이블 생성
CREATE TABLE testTBL4 (
    id INT,
    Fname VARCHAR(50),
    Lname VARCHAR(50)
);

-- 다른 테이블 데이터 삽입
INSERT INTO testTBL4
```

```
SELECT emp_no, first_name, last_name
FROM employees.employees;
```

예시 5) 테이블 정의와 데이터를 함께 생성

- `CREATE TABLE ... SELECT` 구문을 사용하여 새로운 테이블을 생성하고 데이터를 복사
- 스키마(열 구조)를 명시적으로 정의하지 않아도 자동으로 생성됨

```
CREATE TABLE testTBL5
(
SELECT emp_no, first_name, last_name
FROM employees.employees;
)
```

(참고) 오류 무시하고 계속 삽입되도록 하기

- PK가 중복되면, 중복되는 데이터는 무시하고 삽입되도록 하고 싶은 경우, `INSERT IGNORE INTO` 를 사용

```
INSERT IGNORE INTO memberTBL VALUES ('KHD', '강후덜', '미국');
```

- 만약 새롭게 삽입되는 데이터로 수정하고 싶은 경우, `ON DUPLICATE KEY UPDATE` 를 사용

```
INSERT INTO memberTBL VALUES ('KHD', '강후덜', '미국')
ON DUPLICATE KEY UPDATE userName='강후덜', addr='미국';
```

AUTO_INCREMENT

- 자동으로 1부터 증가하는 값을 입력하는 키워드
- 특정 열(column)을 `AUTO_INCREMENT` 로 설정할 경우, 반드시 **PRIMARY KEY**(기본키) 또는 **UNIQUE**(고유값)으로 설정
- 데이터 형식은 숫자형 열에만 사용 가능
- `AUTO_INCREMENT` 로 지정된 열은 `INSERT` 문에서 **NULL** 값을 지정하면 자동으로 값이 입력됨

예시

```
USE cookDB;

CREATE TABLE testTBL2 (
    id INT AUTO_INCREMENT PRIMARY KEY,
    userName CHAR(3),
    age INT
);

--id 부분은 전부 NULL 입력
INSERT INTO testTBL2 VALUES (NULL, '에디', 15);
INSERT INTO testTBL2 VALUES (NULL, '포비', 12);
INSERT INTO testTBL2 VALUES (NULL, '통통이', 11);

SELECT * FROM testTBL2;
```

(참고) AUTO_INCREMENT 시작값 및 증가 단위 설정

```
USE cookDB;

-- 테이블 생성
CREATE TABLE testTBL3 (
    id INT AUTO_INCREMENT PRIMARY KEY,
    userName CHAR(3),
    age INT
);

-- AUTO_INCREMENT 초기값 설정 (1000부터 시작)
ALTER TABLE testTBL3 AUTO_INCREMENT = 1000;

-- AUTO_INCREMENT 증가값 설정 (3씩 증가)
SET @@auto_increment_increment = 3;

-- 데이터 삽입
INSERT INTO testTBL3 VALUES (NULL, '우디', 20);
INSERT INTO testTBL3 VALUES (NULL, '버즈', 18);
INSERT INTO testTBL3 VALUES (NULL, '제시', 19);

-- 결과 확인
SELECT * FROM testTBL3;
```

UPDATE

- UPDATE 는 테이블에 입력된 값을 수정할 때 사용하는 SQL 명령어
- 특정 조건(WHERE)에 맞는 데이터만 업데이트
- 기본 형식

```
UPDATE 테이블이름
SET 열1 = 값1, 열2 = 값2, ...
WHERE 조건;
```

예시 1) 특정 값 수정

```
-- Kyoichi라는 Fname을 가진 사람의 Lname을 '없음'으로 설정
UPDATE testTBL4
SET Lname = '없음'
WHERE Fname = 'Kyoichi';
```

예시 2) 전체 테이블 내용 수정

- 테이블의 모든 행을 수정하고자 할 때는 WHERE 절을 생략
- 아래 예제에서는 buyTBL 테이블의 price 열 값을 1.5배로 수정

```
UPDATE buyTBL
SET price = price * 1.5;
```

DELETE

- `DELETE` 는 테이블에서 데이터를 **행** 단위로 삭제하는 SQL 명령어
- 특정 조건에 맞는 데이터만 삭제하려면 `WHERE` 절을 사용
- 기본 형식

```
DELETE FROM 테이블이름 WHERE 조건;
```

예시 1) 특정 조건으로 일부 데이터 삭제

- 특정 조건(`Fname = 'Aamer'`)에 맞는 데이터를 삭제
- 조건이 없으면 모든 데이터가 삭제되므로 주의 필요

```
DELETE FROM testTBL4 WHERE Fname = 'Aamer';
```

예시 2) DELETE 문에서 WHERE 절 생략

- `WHERE` 절을 생략하면 테이블에 저장된 **전체** 데이터가 삭제된다
- 단, 테이블 구조(스키마)는 남아있음

```
DELETE FROM testTBL4 WHERE Fname = 'Aamer';
```

DELETE, DROP, TRUNCATE 비교

세 가지 구문 비교

- `DELETE` 문: 특정 데이터 삭제. 실행 시간이 길어질 수 있음.
- `DROP` 문: 테이블 자체 삭제. 테이블 구조 및 데이터 모두 제거됨.
- `TRUNCATE` 문 : 데이터만 삭제. 테이블 구조는 남음. 실행 속도가 빠름.

```
DELETE FROM bigTBL1;
DROP TABLE bigTBL2;
TRUNCATE TABLE bigTBL3;
```

실행 시간 관점에서

- 대용량 테이블 전체 내용을 삭제할 때 테이블 자체가 필요 없는 경우에는 `DROP` 문 사용
- 테이블의 구조를 남겨놓고 싶은 경우에는 `TRUNCATE` 문으로 삭제

윈도우 함수

- 테이블의 행과 행 간 관계를 쉽게 정의하기 위해 사용.
- `OVER` 절이 포함된 함수.
- 함께 사용되는 함수 예시
 - 윈도우 함수와 함께 사용되는 집계함수
 - `AVG()` : 평균
 - `COUNT()` : 개수
 - `MAX()` : 최대값
 - `MIN()` : 최소값
 - `STDDEV()` : 표준편차

- `SUM()` : 합계
- `VARIANCE()` : 분산
- 윈도우 함수와 함께 사용되는 비집계 함수 : 행 간 비교나 순위 산출 등에서 사용
 - `CUME_DIST()` : 누적 백분위
 - `DENSE_RANK()` : 연속 순위
 - `FIRST_VALUE()` : 첫 번째 값
 - `LAG()` : 이전 행의 값
 - `LAST_VALUE()` : 마지막 값
 - `LEAD()` : 다음 행의 값
 - `NTH_VALUE()` : N번째 값
 - `NTILE()` : 그룹 나누기
 - `PERCENT_RANK()` : 백분위 순위
 - `RANK()` : 순위
 - `ROW_NUMBER()` : 행 번호

ROW_NUMBER

- `ROW_NUMBER()` 윈도우 함수를 사용해 특정 열을 기준으로 순위를 매김
 - `ORDER BY` 로 키가 큰 순으로 정렬.
 - 순위는 `ROW_NUMBER()` 함수로 계산.

예시 1) 키가 큰 순으로 정렬하기

```
SELECT ROW_NUMBER() OVER (ORDER BY height DESC) AS "키큰순위",
       userName,
       addr,
       height
FROM userTBL;
```

예시 2) 키가 같은 경우 이름의 가나다순으로 정렬

```
SELECT ROW_NUMBER() OVER(ORDER BY height DESC, userName ASC) AS "키큰순위",
       userName, addr, height
FROM userTBL;
```

DENSE_RANK

- 동일한 등수를 가진 경우, 동일 등수로 처리 후 다음 등수는 그대로 이어지도록 처리
- Ex) 1위 - 2위 - 3위 - 3위 - 4위 - ...

예시 1) 키가 같은 경우 동일한 등수 처리

```
SELECT DENSE_RANK() OVER(ORDER BY height DESC) AS "키큰순위",
       userName, addr, height
FROM userTBL;
```

RANK

- 동일한 값/순위를 가진 경우 동일 등수 처리 후, 다음 순위를 건너뛴

- Ex) 1위 - 2위 - 3위 - 3위 - 5위 - ...

예시 1) 동일 등수 처리 후, 동일 등수 ROW 수를 고려해서 순위 매기기

```
SELECT RANK() OVER(ORDER BY height DESC) AS "키큰순위",
       userName, addr, height
FROM userTBL;
```

NTILE

- 크기에 따라서 N개의 그룹으로 분할시켜주는 함수

예시 1) 키의 크기에 따라 3개의 반으로 분리

```
SELECT NTILE(3) OVER(ORDER BY height DESC) AS "반번호",
       userName, addr, height
FROM userTBL;
```

LEAD

- LEAD() 윈도우 함수를 사용하여 현재 행과 다른 행들간의 연산을 수행할 수 있음
- Ex) 현재 행의 키(height)와 다음 행의 키 차이를 계산

예시 1) 키 큰 순으로 정렬 후 다음 사람과 키 차이 구하기

```
SELECT userName,
       addr,
       height AS "키",
       height - LEAD(height, 1, 0) OVER (ORDER BY height DESC) AS "다음 사람과 키 차이"
FROM userTBL;
```

- LEAD(height, 1, 0):
 - 현재 행의 다음 행 값을 가져옴
 - 세 번째 인수 0 은 다음 행이 없을 경우 반환할 기본값
- OVER (ORDER BY height DESC):
 - 키 순으로 정렬하여 윈도우 함수 적용

FIRST_VALUE

- FIRST_VALUE() 윈도우 함수를 사용하여 Partition별 가장 값과 Row별 값들간의 연산을 수행

예시 1) 지역별 가장 키가 큰 사람과의 키 차이 구하기

- FIRST_VALUE() 윈도우 함수를 사용하여 지역별 가장 키가 큰 사람의 키와 나머지 사람들의 키 차이를 계산.
- 지역별 그룹을 나누고(PARTITION BY addr), 키 순으로 정렬하여 계산.

```
SELECT addr,
       userName,
       height AS "키",
```

```
height - FIRST_VALUE(height) OVER (PARTITION BY addr ORDER BY height DESC) AS "지역별 키",
FROM userTBL;
```

- **FIRST_VALUE(height)**:
 - 지역별 그룹 내에서 가장 큰 키를 반환.
- **PARTITION BY addr**:
 - 지역별 그룹을 나눔.

CUME_DIST

- 누적 백분율을 계산
- 마찬가지로 PARTITION을 같이 이용해서 특정 기준 안에서의 누적 백분율로 나타낼 수도 있음

예시 1) 같은 지역 회원과 비교하여 키가 크거나 같은 사람이 전체의 몇 %인지 계산

- **CUME_DIST()** 윈도우 함수를 사용해 누적 백분율을 계산.
- 지역별로 그룹을 나눈 후(**PARTITION BY addr**), 키 순으로 정렬하여 누적 백분율 계산.
- 계산된 백분율에 100을 곱해 백분율로 표시.

```
SELECT addr,
       userName,
       height AS "키",
       (CUME_DIST() OVER (PARTITION BY addr ORDER BY height DESC)) * 100 AS "누적인원 백분율%"
FROM userTBL;
```

- **CUME_DIST()**:
 - 현재 행이 그룹에서 차지하는 누적 백분율을 계산.
 - (현재 행 순위) ÷ (전체 행 수).
- **PARTITION BY addr**:
 - 지역별로 데이터를 그룹화.

피벗

- 한 열에 포함된 여러 값을 여러 열로 변환하여 출력.
- 필요한 경우 집계 함수를 사용해 데이터를 요약.
- 결과로 피벗 테이블 생성.
- 기본 형식

```
SELECT *
FROM (
    SELECT column1, column2, value_column
    FROM source_table
) AS SourceTable
PIVOT (
    AGGREGATE_FUNCTION(value_column)
    FOR column2 IN ([Value1], [Value2], [Value3])
) AS PivotTable;
```

- 한편 **GROUP BY** 절을 이용해서 피벗 테이블을 만드는 것도 가능

```

SELECT uName,
       SUM(CASE WHEN season = '봄' THEN amount END) AS '봄',
       SUM(CASE WHEN season = '여름' THEN amount END) AS '여름',
       SUM(CASE WHEN season = '가을' THEN amount END) AS '가을',
       SUM(CASE WHEN season = '겨울' THEN amount END) AS '겨울'
FROM pivotTest
GROUP BY uName;

```

CTE (Common Table Expression)

WITH

- SQL에서 임시 결과 집합을 정의하고 이를 쿼리에서 재사용.
- 종류:
 1. 비재귀적(Non-Recursive) CTE:
 - 단순 쿼리의 결과를 임시 테이블처럼 활용.
 2. 재귀적(Recursive) CTE:
 - 자기 자신을 참조하여 반복 작업 수행.

예시 1) 비재귀적 CTE

```

WITH SalesCTE AS (
    SELECT productID, SUM(salesAmount) AS TotalSales
    FROM Sales
    GROUP BY productID
)

SELECT *
FROM SalesCTE
WHERE TotalSales > 1000;

```

- 설명:
 - SalesCTE 라는 이름으로 임시 결과 집합 생성.
 - 조건에 맞는 데이터를 최종 쿼리에서 사용.

예시 2) 재귀적 CTE

```

WITH RECURSIVE EmployeeHierarchy AS (
    -- Anchor Member: 첫 번째 단계의 데이터를 선택
    SELECT employeeID, managerID, 1 AS Level
    FROM Employees
    WHERE managerID IS NULL -- 최상위 관리자 (root)

    UNION ALL

    -- Recursive Member: 이전 단계의 데이터를 기반으로 새로운 데이터를 선택
    SELECT e.employeeID, e.managerID, eh.Level + 1
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh ON e.managerID = eh.employeeID
)

```

```
SELECT *  
FROM EmployeeHierarchy;
```

작동 원리

5. Anchor Member:

- 최초 데이터 생성:
 - WHERE managerID IS NULL 조건에 따라 최상위 관리자(루트 노드)를 찾음.
 - 예: (employeeID: 1, managerID: NULL, Level: 1).

6. Recursive Member:

- 반복적으로 자신을 참조:
 - EmployeeHierarchy에서 현재 노드의 employeeID가 다른 노드의 managerID와 매칭되는 행을 찾음.
 - 찾은 행에 대해 Level 값을 1씩 증가시켜 새로운 데이터를 생성.

7. 반복 종료:

- 더 이상 연결된 데이터가 없으면 재귀 종료.
- 기본적으로 MySQL에서는 1,000단계까지 재귀 허용(필요 시 설정 변경 가능).

그 외의 사용 예시

- 8. 조직도 : 계층적 데이터(예: 부서와 직원 관계) 출력.
- 9. 디렉터리 구조 : 폴더 및 파일 구조 출력.
- 10. 그래프 탐색 : 노드와 엣지 데이터의 계층적 탐색.

예시 3) 각 지역별 가장 큰 키들의 평균

```
WITH cte_userTBL(addr, maxHeight) AS (  
    SELECT addr, MAX(height)  
    FROM userTBL  
    GROUP BY addr  
)  
  
SELECT AVG(maxHeight * 1.0) AS '각 지역별 최고키의 평균'  
FROM cte_userTBL;
```

11. CTE 정의:

- cte_userTBL이라는 이름의 Common Table Expression(CTE)을 생성.
- addr(지역)별 최고 키(MAX(height))를 계산.

12. 최종 쿼리:

- AVG() 함수를 사용하여 CTE에서 계산된 각 지역별 최고 키(maxHeight)의 평균을 구함.
- * 1.0 은 정수 데이터를 실수로 변환해 정확한 평균 계산을 보장.

(참고) CTE는 중첩 CTE로 사용 가능

- 한 CTE에서 정의된 결과를 다른 CTE에서 참조 가능.
- 하지만 상위 CTE(예: CCC)는 하위 CTE(예: AAA, BBB)에서 참조할 수 없음.
- CTE는 정의된 순서대로 참조 가능.

```
WITH  
AAA (컬럼들) AS (  
    -- AAA의 쿼리문  
    SELECT ...  
) ,  
BBB (컬럼들) AS (  
    -- BBB의 쿼리문: AAA를 참조할 수 있음
```

```

SELECT ...
FROM AAA
),
CCC (칼럼들) AS (
  -- CCC의 쿼리문: AAA와 BBB를 참조할 수 있음
  SELECT ...
  FROM BBB
)

-- 최종 쿼리: AAA, BBB, 또는 CCC를 사용
SELECT *
FROM AAA
UNION ALL
SELECT *
FROM BBB
UNION ALL
SELECT *
FROM CCC;

```

- CTE는 순차적으로 참조 가능:
 - BBB는 AAA 참조 가능.
 - CCC는 AAA와 BBB 참조 가능.
 - 역참조는 불가능 (예: AAA에서 CCC를 참조).

(5주차) 데이터 형식과 내장 함수

데이터 형식

숫자데이터

데이터 형식	바이트 수	숫자 범위	설명 및 특징
BIT(n)	N/8	1~64비트 표현	비트 필드 데이터를 저장. <code>b'0000'</code> 형식으로 저장하며 최대 64비트까지 사용 가능.
TINYINT	1	-128 ~ 127	작은 정수 범위를 저장.
BOOL / BOOLEAN	1	-128 ~ 127	TINYINT(1)과 동일. 0은 FALSE, 1은 TRUE로 취급.
SMALLINT	2	-32,768 ~ 32,767	중간 크기의 정수 범위를 저장.
MEDIUMINT	3	-8,388,608 ~ 8,388,607	일반적인 정수보다 더 큰 정수를 저장할 때 사용.
INT / INTEGER	4	약 -21억 ~ 21억	기본적인 정수형 데이터 타입. 가장 널리 사용됨.
BIGINT	8	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807	매우 큰 정수를 저장 가능. 예를 들어, 금융 데이터나 초대형 ID 저장 시 유용.
FLOAT	4	약 $\pm 3.40E-38$ ~ $\pm 1.17E+38$	부동소수점으로 저장. 소수점 이하 7자리까지 정확한 값을 저장 가능.
DOUBLE / REAL	8	약 $\pm 1.22E-308$ ~ $\pm 1.79E+308$	부동소수점으로 저장. 소수점 이하 15자리까지 정확한 값을 저장 가능.

데이터 형식	바이트 수	숫자 범위	설명 및 특징
DECIMAL(m, d)	5~17	$-10^{38}+1 \sim 10^{38}-1$	고정소수점 데이터 타입. 전체 자릿수(m)와 소수점 이하 자릿수(d)를 지정 가능.
			예: DECIMAL(5, 2) 는 최대 999.99 까지 저장 가능.
DEC / NUMERIC(m, d)	5~17	$-10^{38}+1 \sim 10^{38}-1$	DECIMAL 과 동일.

문자 데이터

데이터 형식	바이트 수	설명
CHAR(n)	1~255	고정 길이 문자 저장. n에 지정된 크기만큼 공간 사용. 짧으면 공백으로 채움.
VARCHAR(n)	1~65535	가변 길이 문자 저장. 값의 길이에 따라 공간 사용. 길이 정보 추가 저장.
BINARY(n)	1~255	고정 길이 이진 데이터 저장. 짧으면 0으로 패딩.
VARBINARY(n)	1~65535	가변 길이 이진 데이터 저장. 값의 길이에 따라 공간 사용.
TINYTEXT	1~255	최대 255바이트 크기의 텍스트 저장.
TEXT	1~65535	최대 65535바이트 크기의 텍스트 저장.
MEDIUMTEXT	1~16,777,215	최대 16MB 크기의 텍스트 저장.
LONGTEXT	1~4,294,967,295	최대 4GB 크기의 텍스트 저장.
TINYBLOB	1~255	최대 255바이트 크기의 이진 데이터 저장.
BLOB	1~65535	최대 65535바이트 크기의 이진 데이터 저장.
MEDIUMBLOB	1~16,777,215	최대 16MB 크기의 이진 데이터 저장.
LOBLOB	1~4,294,967,295	최대 4GB 크기의 이진 데이터 저장.
ENUM	1 또는 2	미리 정의된 값 중 하나 선택. 최대 65535개의 값 정의 가능.
SET	1, 2, 3, 4, 8	미리 정의된 값 중 여러 개 선택 가능. 최대 64개의 값 지정 가능.

날짜와 시간 데이터

데이터 형식	바이트 수	설명
DATE	3	YYYY-MM-DD 형식으로 날짜 저장. 저장 범위: 1001-01-01 ~ 9999-12-31.
TIME	3	HH:MM:SS 형식으로 시간 저장. 저장 범위: -838:59:59 ~ 838:59:59.
DATETIME	8	YYYY-MM-DD HH:MM:SS 형식으로 날짜와 시간 저장. 저장 범위: 1001-01-01 00:00:00 ~ 9999-12-31 23:59:59.
TIMESTAMP	4	YYYY-MM-DD HH:MM:SS 형식으로 날짜와 시간 저장. - 저장 범위: 1001-01-01 00:00:00 ~ 9999-12-31 23:59:59. - UTC 시간대로 변환하여 저장. 시스템의 time_zone 설정에 영향을 받음
YEAR	1	YYYY 형식으로 연도 저장. 저장 범위: 1901 ~ 2155.

(참고) 차이점 요약 - DATETIME vs. TIMESTAMP

데이터 형식	주요 차이점
DATETIME	시스템 시간대에 영향받지 않음.
TIMESTAMP	UTC 시간대로 변환 후 저장. 시스템 <code>time_zone</code> 설정에 따라 값이 변환되어 반환됨.

(참고) CAST 함수와의 활용

- `CAST()` 함수데이터 형식을 명시적으로 변환하는 SQL 함수.
- 위 예제에서는 `'2020-10-19 12:35:29.123'` 문자열을 다양한 날짜/시간 형식으로 변환.

```
SELECT CAST('2020-10-19 12:35:29.123' AS DATE) AS 'DATE';
SELECT CAST('2020-10-19 12:35:29.123' AS TIME) AS 'TIME';
SELECT CAST('2020-10-19 12:35:29.123' AS DATETIME) AS 'DATETIME';
```

데이터 형식 변환 함수

CAST, CONVERT

- 일반적으로 `CAST()`, `CONVERT()` 사용
- 두 함수 기능은 거의 비슷
- 기본 형태

```
CAST(expression AS 데이터형식 [(길이)])
CONVERT(expression, 데이터형식 [(길이)])
```

예시 1)

```
SELECT CAST(AVG(amount) AS SIGNED INTEGER) AS '평균 구매 개수' FROM buyTBL;
SELECT CONVERT(AVG(amount), SIGNED INTEGER) AS '평균 구매 개수' FROM buyTBL;
```

결과 차이

쿼리	결과 값
<code>SELECT AVG(amount) AS '평균 구매 개수'</code>	2.9167 (실수)
<code>SELECT CAST(AVG(amount) AS SIGNED INTEGER)</code>	3 (정수)
<code>SELECT CONVERT(AVG(amount), SIGNED INTEGER)</code>	3 (정수)

예시 2) 다양한 구분자(\$, /, %, @)를 날짜 형식(-)으로도 변경

```
SELECT CAST('2020$12$12' AS DATE);
SELECT CAST('2020/12/12' AS DATE);
SELECT CAST('2020%12%12' AS DATE);
SELECT CAST('2020@12@12' AS DATE);
```

- 전부 결과는 2020-12-12 반환

예시 3) 단가와 수량을 곱한 실제 금액 출력

```
SELECT num,  
       CONCAT(CAST(price AS CHAR(10)), 'X', CAST(amount AS CHAR(4)), '=') AS '단가×수량',  
       price * amount AS '구매액'  
FROM buyTBL;
```

- `CONCAT`: 문자열을 연결하여 출력.
- `CAST`: 숫자 데이터를 문자로 변환.
- `price * amount`: 단가와 수량을 곱해 실제 구매 금액 계산.

(참고) CAST와 CONVERT

- 두 함수는 기능적으로 거의 동일하지만, 표준 SQL에서는 **CAST**가 더 권장됨.
- **CONVERT**는 MySQL 외에도 SQL Server에서 지원되며, 일부 구현에서 추가 옵션 제공.

FORMAT_DATETIME

- 날짜와 시간을 사용자가 원하는 형식으로 출력할 수 있도록 포매팅하는 함수
- 주로 **MySQL 8.0.19** 이상에서 제공
- 사용자 지정 형식에 따라 날짜 및 시간 값을 출력하는 데 유용
- 기본 형태

```
FORMAT_DATETIME(datetime, format)
```

예시 1) 표준 날짜 형식

```
SELECT FORMAT_DATETIME('2025-01-24 21:15:30', '%Y-%m-%d %H:%i:%s') AS '표준 날짜 형식';
```

결과: 2025-01-24 21:15:30

(참고) 다양한 포맷 지정

포맷 문자열	의미	예제 결과
%Y	4자리 연도	2025
%m	2자리 월	01
%d	2자리 일	24
%H	24시간제 시간	21
%i	분	15
%S	초	30
%a	요일 약자 (영문)	Fri (금요일)
%W	요일 전체 이름 (영문)	Friday
%p	오전/오후 표시	PM
%M	월 이름 (영문)	January

암시적인 형 변환

- **CAST**나 **CONVERT** 함수 없이 MySQL이 데이터의 형식을 자동으로 변환하여 처리하는 방식
- 연산에 사용된 데이터 유형에 따라 자동으로 숫자 ↔ 문자 ↔ 논리 간 변환이 일어남

예시) 여러가지 자동 형변환

```
-- 1. 문자열과 숫자 덧셈
SELECT '100' + '200'; -- 문자열을 숫자로 변환 후 계산

-- 2. CONCAT 함수로 문자열 연결
SELECT CONCAT('100', '200'); -- 숫자가 문자열로 처리되어 연결

-- 3. CONCAT 함수로 숫자 + 문자열
SELECT CONCAT(100, '200'); -- 숫자 100이 문자열로 변환되어 연결

-- 4. 숫자와 문자열 비교
SELECT 1 > '3mega'; -- 정수인 3으로 변환한 후 비교

-- 5. 다른 문자열과 숫자 비교
SELECT 4 > '3MEGA'; -- 정수인 3으로 변환한 후 비교

-- 6. 숫자와 혼합된 문자열 비교
SELECT 0 = 'mega3'; -- 문자가 0으로 변환된 후 비교
```

쿼리 결과

쿼리 번호	입력값/함수	결과값	처리 방식
1	'100' + '200'	300	문자열 → 숫자로 변환 후 덧셈
2	CONCAT('100', '200')	100200	문자열 그대로 연결
3	CONCAT(100, '200')	100200	숫자 100이 문자열로 변환 후 연결
4	1 > '3mega'	0 (False)	'mega' → 숫자 3 변환 후 비교
5	4 > '3MEGA'	1 (True)	'MEGA' → 숫자 3 변환 후 비교
6	0 = 'mega3'	1 (True)	'mega3' → 숫자 0 변환 후 0과 비교

변수의 선언과 활용

SET

- 변수 선언 및 값 대입
- 기본 형태

```
SET @변수이름 = 값; -- 변수 선언 및 값 대입
SELECT @변수이름; -- 변수 값 출력
```

예시 1) 기본 구문

```
SET @myVar1 = 5;
SET @myVar2 = 3;
SET @myVar3 = 4.25;
SET @myVar4 = 'MC 이름==> ';
```

```
SELECT @myVar1;           -- 변수 출력
SELECT @myVar2 + @myVar3; -- 변수 값 계산
SELECT @myVar4, userName  -- 조건에 따른 값 출력
FROM userTBL
WHERE height > 180;
```

PREPARE & EXECUTE

- 쿼리를 준비해 PREPARE에 준 다음, EXECUTE을 통해서 실행
- 앞서 설정한 변수 선언 사용 가능

예시 1) 선언한 변수를 활용해 PREPARE - EXECUTE 하기

```
SET @myVar1 = 3;

PREPARE myQuery FROM
    'SELECT userName, height FROM userTBL ORDER BY height LIMIT ?';

EXECUTE myQuery USING @myVar1;
```

13. 변수 선언 및 대입:

- SET 구문을 사용하여 변수에 값을 할당.
- 다양한 데이터 타입(정수, 실수, 문자열)을 저장 가능.

14. 변수 출력:

- SELECT 구문을 통해 변수 값을 출력하거나 계산 가능.

15. PREPARE와 EXECUTE:

- 동적 SQL 실행에 변수 활용.
- PREPARE 는 쿼리를 준비, EXECUTE 는 실행.

(내장 함수) 제어 흐름 함수

IF

- IF(수식, 참, 거짓)
- 수식이 참이면 두 번째 인수를 반환, 거짓이면 세 번째 인수를 반환.

예시)

```
SELECT IF(100>200, '참이다', '거짓이다');
```

- 결과: '거짓이다'

IFNULL

- IFNULL(수식1, 수식2)
 - 수식1 이 NULL이 아니면 수식1 반환
 - 수식1 이 NULL이면 수식2 반환

예시)

```
SELECT IFNULL(NULL, '널이군요'), IFNULL(100, '널이군요');
```

- 결과:
 - 첫 번째 쿼리: '널이군요'
 - 두 번째 쿼리: 100

NULLIF

- NULLIF(수식1, 수식2)
 - 수식1 과 수식2 가 같으면 NULL 반환
 - 다르면 수식1 반환

예시)

```
SELECT NULLIF(100, 100), NULLIF(200, 100);
```

- 결과:
 - 첫 번째 쿼리: NULL
 - 두 번째 쿼리: 200

CASE ... WHEN ... ELSE ... END

- CASE는 연산자(operator)로 다중 조건 분기에 사용.
- 조건에 맞는 WHEN 의 결과 반환. 조건이 없으면 ELSE 반환.

예시)

```
SELECT CASE 10
      WHEN 1 THEN '일'
      WHEN 5 THEN '오'
      WHEN 10 THEN '십'
      ELSE '모름'
END;
```

(내장 함수) 문자열 함수

ASCII & CHAR

- ASCII(문자) : 주어진 문자의 아스키코드 값을 반환.
- CHAR(숫자) : 숫자에 해당하는 아스키코드 문자 반환.

예시)

```
SELECT ASCII('A'), CHAR(65);
```

쿼리	결과
SELECT ASCII('A');	65
SELECT CHAR(65);	A

BIT_LENGTH, CHAR_LENGTH, LENGTH

- `BIT_LENGTH(문자열)` : 문자열의 비트 길이 반환.
- `CHAR_LENGTH(문자열)` : 문자열의 문자 개수 반환.
- `LENGTH(문자열)` : 문자열의 바이트 길이 반환 (UTF-8 기준 한글은 3바이트).

예시)

```
SELECT BIT_LENGTH('abc');      -- 'abc'의 비트 길이 반환
SELECT CHAR_LENGTH('abc');     -- 'abc'의 문자 개수 반환
SELECT LENGTH('abc');          -- 'abc'의 바이트 길이 반환
SELECT BIT_LENGTH('가나다');   -- '가나다'의 비트 길이 반환
SELECT CHAR_LENGTH('가나다');  -- '가나다'의 문자 개수 반환
SELECT LENGTH('가나다');       -- '가나다'의 바이트 길이 반환
```

쿼리	결과
SELECT BIT_LENGTH('abc');	24
SELECT CHAR_LENGTH('abc');	3
SELECT LENGTH('abc');	3
SELECT BIT_LENGTH('가나다');	72
SELECT CHAR_LENGTH('가나다');	3
SELECT LENGTH('가나다');	9

CONCAT, CONCAT_WS

- `CONCAT(문자열1, 문자열2, ...)` : 여러 문자열을 이어붙임.
- `CONCAT_WS(구분자, 문자열1, 문자열2, ...)` : 구분자를 포함하여 문자열을 이어붙임.

예시)

```
SELECT CONCAT('2020', '01', '01');      -- 문자열 이어붙임
SELECT CONCAT_WS('/', '2020', '01', '01'); -- '/' 구분자로 문자열 이어붙임
```

SELECT CONCAT('2020', '01', '01');	20200101
SELECT CONCAT_WS('/', '2020', '01', '01');	2020/01/01

FORMAT

- 숫자를 소수점 이하 지정된 자리수까지 표시하고, 1,000단위로 쉼표(,)를 추가

예시)

```
SELECT FORMAT(123456.789, 2); -- 숫자를 쉼표와 함께 포맷
```

결과 : 123,456.79

INSERT

- 문자열의 특정 위치에서 길이만큼 문자를 삭제하고, 새로운 문자열 삽입.

예시)

```
SELECT INSERT('abcdefghi', 3, 4, '@@@'); -- 'abc'에서 시작해 4글자를 삭제하고 '@@@' 삽입
```

결과 : ab@@@ghi

LOWER와 UPPER

- `LOWER(문자열)` : 입력 문자열을 소문자로 변환
- `UPPER(문자열)` : 입력 문자열을 대문자로 변환

예시)

```
SELECT LOWER('ABC'); -- 'ABC'를 소문자로 변환  
SELECT UPPER('abc'); -- 'abc'를 대문자로 변환
```

결과 : abc, ABC

LTRIM, RTRIM

- `LTRIM(문자열)` : 문자열 왼쪽 공백 제거.
- `RTRIM(문자열)` : 문자열 오른쪽 공백 제거.

예시)

```
SELECT LTRIM('   공백제거'); -- 왼쪽 공백 제거  
SELECT RTRIM('공백제거   '); -- 오른쪽 공백 제거
```

REPLACE

- 문자열에서 특정 부분을 새로운 문자열로 교체

예시)

```
SELECT REPLACE('I ATE POPCORN', 'POPCORN', 'CHIPS'); -- 'POPCORN'을 'CHIPS'로 교체
```

REVERSE

- 입력된 문자열의 순서를 거꾸로 반환

예시)

```
SELECT REVERSE('MySQL'); -- 문자열 순서를 거꾸로 반환
```

결과 : LQSyM

SUBSTRING

- 문자열의 특정 위치부터 지정된 길이만큼

예시)

```
SELECT SUBSTRING('대한민국만세', 3, 2); -- 3번째 위치부터 2글자 반환
```

결과 : 민국

(내장 함수) 수학 함수

POW(숫자1, 숫자2), SQRT(숫자)

- POW(숫자1, 숫자2) : 숫자1을 숫자2만큼 거듭 제공한 값을 반환.
- SQRT(숫자) : 숫자의 제곱근 반환.

예시)

```
SELECT POW(2, 3), SQRT(9);
```

결과 : 8, 3

RAND

- 0 이상 1 미만의 난수를 반환.
- 난수를 이용해 특정 범위의 값을 생성 가능.

```
SELECT  
  RAND(), -- 0-1 사이의 숫자  
  FLOOR(1 + (RAND() * (6 - 1))); -- 1-6 사이의 숫자 (주사위)
```

(내장 함수) 날짜/시간 함수

ADDDATE(날짜, 차이), SUBDATE(날짜, 차이)

- ADDDATE는 날짜를 기준으로 차이를 더하거나 뺀 날짜를 반환
- SUBDATE는 날짜에서 차이를 빼거나 더한 날짜를 반환

예시)

```
SELECT ADDDATE('2020-01-01', INTERVAL 31 DAY), ADDDATE('2020-01-01', INTERVAL 1 MONTH);  
SELECT SUBDATE('2020-01-01', INTERVAL 31 DAY), SUBDATE('2020-01-01', INTERVAL 1 MONTH);
```

결과:

Query	Result
ADDDATE('2020-01-01', INTERVAL 31 DAY)	2020-02-01
ADDDATE('2020-01-01', INTERVAL 1 MONTH)	2020-02-01

Query	Result
SUBDATE('2020-01-01', INTERVAL 31 DAY)	2019-12-01
SUBDATE('2020-01-01', INTERVAL 1 MONTH)	2019-12-01

ADDTIME(날짜/시간, 시간), SUBTIME(날짜/시간, 시간)

- ADDTIME은 날짜/시간에 특정 시간을 더함.
- SUBTIME은 날짜/시간에서 특정 시간을 뺌.

예시)

```
SELECT ADDTIME('2020-01-01 23:59:59', '1:1:1'), ADDTIME('15:00:00', '2:10:10');
SELECT SUBTIME('2020-01-01 23:59:59', '1:1:1'), SUBTIME('15:00:00', '2:10:10');
```

결과:

Query	Result
ADDTIME('2020-01-01 23:59:59', '1:1:1')	2020-01-02 01:01:00
ADDTIME('15:00:00', '2:10:10')	17:10:10
SUBTIME('2020-01-01 23:59:59', '1:1:1')	2020-01-01 22:58:58
SUBTIME('15:00:00', '2:10:10')	12:49:50

YEAR(날짜), MONTH(날짜), DAY(날짜), HOUR(시간), MINUTE(시간), SECOND(시간), MICROSECOND(시간)

- 날짜 또는 시간에서 연, 월, 일, 시, 분, 초, 밀리초를 구함.

예시)

```
SELECT YEAR(CURDATE()), MONTH(CURRENT_DATE()), DAYOFMONTH(CURRENT_DATE());
SELECT HOUR(CURTIME()), MINUTE(CURRENT_TIME()), SECOND(CURRENT_TIME()), MICROSECOND(CURREI
```

결과:

Query	Result
YEAR(CURDATE())	2025
MONTH(CURRENT_DATE())	01
DAYOFMONTH(CURRENT_DATE())	25
HOUR(CURTIME())	12
MINUTE(CURRENT_TIME())	30
SECOND(CURRENT_TIME())	45
MICROSECOND(CURRENT_TIME())	123456

DATE(), TIME()

- DATE는 DATETIME 형식에서 연-월-일만 추출.
- TIME은 DATETIME 형식에서 시:분:초만 추출.

예시)

```
SELECT DATE(NOW()), TIME(NOW());
```

결과:

Query	Result
DATE(NOW())	2025-01-25
TIME(NOW())	12:30:45

DATEDIFF(날짜1, 날짜2), TIMEDIFF(날짜1/시간1, 날짜1/시간2)

- DATEDIFF는 날짜1-날짜2의 차이를 반환.
- TIMEDIFF는 시간1-시간2의 차이를 반환.

예시)

```
SELECT DATEDIFF('2023-01-01', NOW()), TIMEDIFF('23:23:59', '12:11:10');
```

결과:

Query	Result	
DATEDIFF('2023-01-01', NOW())	-389	
TIMEDIFF('23:23:59', '12:11:10')	11:12:49	

DAYOFWEEK(날짜), MONTHNAME(날짜), DAYOFYEAR(날짜)

- DAYOFWEEK() 함수는 요일(1: 일~7: 토) 반환.
- MONTHNAME() 함수는 월의 영문(January~December) 반환.
- DAYOFYEAR() 함수는 1년 중 몇 번째 날(1~366)인지 반환.

예시)

```
SELECT DAYOFWEEK(CURDATE()), MONTHNAME(CURDATE()), DAYOFYEAR(CURDATE());
```

결과 :

Query	Result
DAYOFWEEK(CURDATE())	7
MONTHNAME(CURDATE())	January
DAYOFYEAR(CURDATE())	25

LAST_DAY(날짜)

- 입력한 월의 마지막 날짜를 반환.

예시)

```
SELECT LAST_DAY('2020-02-01');
```

결과: 2020-02-29

MAKEDATE(연도, 정수)

- 연도의 첫날부터 정수만큼 지난 날짜를 반환.

예제)

```
SELECT MAKEDATE(2020, 32);
```

결과: 2020-02-01

TIME_TO_SEC(시간)

- 시간을 초 단위로 반환.

예시)

```
SELECT TIME_TO_SEC('12:11:10');
```

결과 : 43870

CURDATE(), CURTIME(), NOW(), SYSDATE()

- CURDATE()는 현재 연-월-일 반환.
- CURTIME()은 현재 시:분:초 반환.
- NOW()와 SYSDATE()는 현재 연-월-일 시:분:초 반환

예시)

```
SELECT CURDATE(), CURTIME(), NOW(), SYSDATE();
```

결과:

Query	Result
CURDATE()	2025-01-25
CURTIME()	12:30:45
NOW()	2025-01-25 12:30:45
SYSDATE()	2025-01-25 12:30:45

(내장 함수) 시스템 정보 함수

USER(), DATABASE()

- 현재 사용자와 현재 선택된 데이터베이스를 반환.

예시)

```
SELECT CURRENT_USER(), DATABASE();
```

FOUND_ROWS()

- 바로 앞의 SELECT 문에서 조회된 행의 개수를 반환.

예시)

```
USE cookDB;  
SELECT * FROM userTBL;  
SELECT FOUND_ROWS();
```

결과 : 10

ROW_COUNT()

- 바로 앞의 INSERT, UPDATE, DELETE 문에서 삽입, 수정, 삭제된 행의 개수를 반환.

예시)

```
USE cookDB;  
UPDATE buyTBL SET price = price * 2;  
SELECT ROW_COUNT();
```

결과 : 12 (예시)

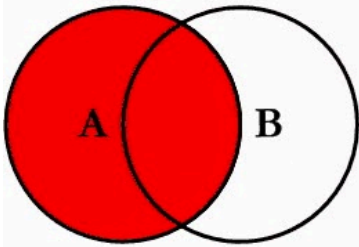
(7주차) 조인과 SQL 프로그래밍

JOIN 기본 개념

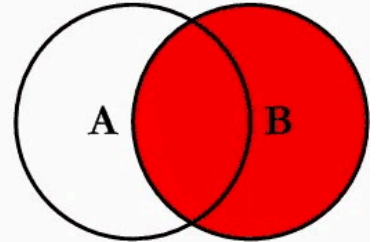
JOIN 기본 개념

- JOIN이란 2개 이상의 테이블을 묶어서 하나의 결과 테이블을 만드는 것
- 1:N 관계 : 한쪽 테이블에는 하나의 값만 존재하고, 그 값과 대응되는 다른 쪽 테이블의 값은 여러 개인 관계
 - Ex)
 - 기업의 직원 테이블과 급여 테이블
 - 학교의 학생 테이블과 학점 테이블

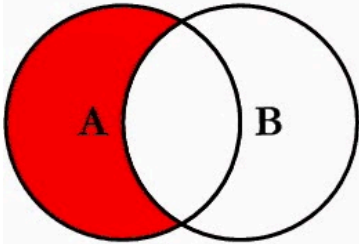
SQL JOINS



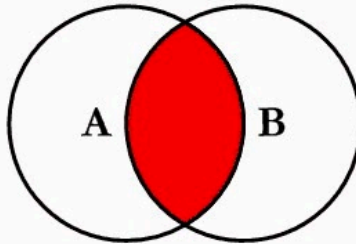
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



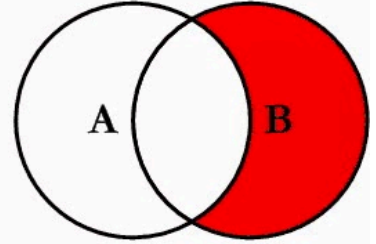
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



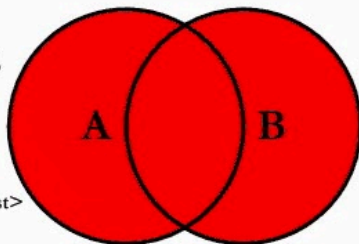
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



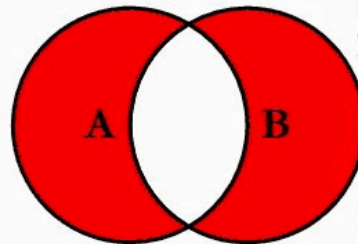
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008

참고) JOIN 용어 정리 (MySQL 기준)

약어/짧은 표현	풀네임/의미	설명
JOIN	INNER JOIN	기본 조인, 공통 값이 있는 행만 반환
LEFT JOIN	LEFT OUTER JOIN	왼쪽 테이블의 모든 행 포함
RIGHT JOIN	RIGHT OUTER JOIN	오른쪽 테이블의 모든 행 포함
FULL JOIN	FULL OUTER JOIN	양쪽 테이블의 모든 행 포함 (MySQL 미지원)
CROSS JOIN	Cartesian Join	모든 조합 반환 (카테시안 곱)
NATURAL JOIN	-	공통 열을 기준으로 자동으로 조인 조건 적용
SELF JOIN	-	같은 테이블 간 조인

INNER JOIN

- 두 테이블에서 동일한 값을 갖는 경우에만 JOIN 수행
- 기본 형태

```
SELECT <열 목록>
FROM <첫 번째 테이블>
INNER JOIN <두 번째 테이블> ON <조인될 조건>
[WHERE 검색조건];`
```

예시 1)

```
USE cookDB;

SELECT *
FROM buyTBL
INNER JOIN userTBL
ON buyTBL.userID = userTBL.userID
WHERE buyTBL.userID = 'KYM';
```

결과:

num	userID	prodName	groupName	price	amount	userName	birthYear	addr	mobile1	m
3	KYM	모니터	전자	200	1	김용만	1967	서울	010	4

작동 원리

- 구매 테이블의 회원 아이디(`buyTBL.userID`)인 KYM을 추출. (WHERE절)
- KYM과 동일한 값을 회원 테이블의 아이디(`userTBL.userID`) 열에서 검색.
- 아이디 KYM을 찾으면 구매 테이블과 회원 테이블의 두 행을 결합(조인).

참고) WHERE 절 생략 시 발생하는 문제

- 아래와 같이 실행할 경우, `userID`라는 컬럼이 여러 개에 존재하므로 에러가 발생한다.

```
SELECT userID, userName, prodName, addr, CONCAT(mobile1, mobile2) AS '연락처'
FROM buyTBL
INNER JOIN userTBL
ON buyTBL.userID = userTBL.userID;
```

결과: Error Code: 1052. Column 'userID' in field list is ambiguous.

- 명시적으로 테이블을 필드에 지정해줘서 해결할 수 있다.

```
SELECT buyTBL.userID, userName, prodName, addr, CONCAT(mobile1, mobile2) AS '연락처'
FROM buyTBL
INNER JOIN userTBL
ON buyTBL.userID = userTBL.userID;
```

참고) JOIN 수행시 컬럼명과 함께 테이블 명 혹은 별칭 사용

- SELECT문에서 각 열의 이름을 명확히 하기 위해 테이블 이름과 열 이름을 함께 작성.

```
SELECT buyTBL.userID, userTBL.userName, buyTBL.prodName, userTBL.addr,
       CONCAT(userTBL.mobile1, userTBL.mobile2) AS '연락처'
FROM buyTBL
INNER JOIN userTBL
ON buyTBL.userID = userTBL.userID;
```

- 테이블 별칭을 활용하여 코드 간결화 및 가독성 향상.

```
SELECT B.userID, U.userName, B.prodName, U.addr, CONCAT(U.mobile1, U.mobile2) AS '연락처'
FROM buyTBL B
INNER JOIN userTBL U
ON B.userID = U.userID;
```

예시 2)

- INNER JOIN과 ORDER BY를 활용하여 전체 회원 구매 목록을 정렬된 데이터로 출력

```
SELECT U.userID, U.userName, B.prodName, U.addr, CONCAT(U.mobile1, U.mobile2) AS '연락처'
FROM userTBL U
INNER JOIN buyTBL B
ON U.userID = B.userID
ORDER BY U.userID;
```

예시 3)

- 한번이라도 구매한 이력이 있는 회원 목록 추출

```
SELECT DISTINCT U.userID, U.userName, U.addr
FROM userTBL U
INNER JOIN buyTBL B
ON U.userID = B.userID
ORDER BY U.userID;
```

예시 4)

- 앞의 예시 3과 동일한 구문이지만, WHERE 절에 EXISTS를 사용해 구매 회원 목록에 적용

```
SELECT U.userID, U.userName, U.addr
FROM userTBL U
WHERE EXISTS (
    SELECT *
    FROM buyTBL B
    WHERE U.userID = B.userID
);
```

OUTER JOIN

- 조인 조건을 만족하지 않는 행까지 포함하여 출력하는 조인
- 기본 형식

```
SELECT <열 목록>
FROM <첫 번째 테이블 (LEFT 테이블)>
<LEFT | RIGHT> OUTER JOIN <두 번째 테이블 (RIGHT 테이블)>
ON <조인될 조건>
[WHERE 검색조건];
```

예시 1) 전체 회원의 구매 기록을 출력하되 구매 기록이 없는 회원도 출력

```
USE cookDB;

SELECT U.userID, U.userName, B.prodName, U.addr, CONCAT(U.mobile1, U.mobile2) AS '연락처'
FROM userTBL U
LEFT OUTER JOIN buyTBL B
ON U.userID = B.userID
ORDER BY U.userID;
```

예시 2) 동일한 결과를 출력하되 RIGHT OUTER JOIN 사용

```
SELECT U.userID, U.userName, B.prodName, U.addr, CONCAT(U.mobile1, U.mobile2) AS '연락처'
FROM buyTBL B
RIGHT OUTER JOIN userTBL U
ON U.userID = B.userID
ORDER BY U.userID;
```

예시 3) 물건을 한 번도 구매한 적이 없는 회원의 목록 조회

```
SELECT U.userID, U.userName, B.prodName, U.addr, CONCAT(U.mobile1, U.mobile2) AS '연락처'
FROM userTBL U
LEFT OUTER JOIN buyTBL B
ON U.userID = B.userID
WHERE B.prodName IS NULL
ORDER BY U.userID;
```

예시 4) 외부 조인을 수행하여 동아리에 가입하지 않은 학생도 출력

```
USE cookDB;
SELECT S.stdName, S.addr, SC.clubName, C.roomNo
FROM stdTBL S
LEFT OUTER JOIN stdclubTBL SC
ON S.stdName = SC.stdName
LEFT OUTER JOIN clubTBL C
ON SC.clubName = C.clubName
ORDER BY S.stdName;
```

예시 5) 동아리를 기준으로 가입 학생을 출력하되, 가입 학생이 한 명도 없는 동아리도 출력

```
SELECT C.clubName, C.roomNo, S.stdName, S.addr
FROM stdTBL S
LEFT OUTER JOIN stdclubTBL SC
ON SC.stdName = S.stdName
RIGHT OUTER JOIN clubTBL C
ON SC.clubName = C.clubName
ORDER BY C.clubName;
```

예제 6) 동아리에 가입하지 않은 학생도 출력하고 학생이 한 명도 없는 동아리도 출력

- UNION을 이용해서 완전 외부 조인을 한 것과 같은 효과 내기

```

SELECT S.stdName, S.addr, SC.clubName, C.roomNo
FROM stdTBL S
LEFT OUTER JOIN stdclubTBL SC
ON S.stdName = SC.stdName
LEFT OUTER JOIN clubTBL C
ON SC.clubName = C.clubName

UNION

SELECT S.stdName, S.addr, SC.clubName, C.roomNo
FROM stdTBL S
LEFT OUTER JOIN stdclubTBL SC
ON S.stdName = SC.stdName
RIGHT OUTER JOIN clubTBL C
ON SC.clubName = C.clubName;

```

CROSS JOIN

- 한쪽 테이블의 모든 행과 다른 쪽 테이블의 모든 행을 조인
- 상호 조인 결과 테이블의 행수는 두 테이블의 행수를 곱한 값.
- 상호 조인은 카티션 곱(cartesian product) 이라고도 함

예시)

```

USE cookDB;
SELECT *
FROM buyTBL
CROSS JOIN userTBL;

```

SELF JOIN

- 자기 자신과 자기 자신을 조인
- 자체 조인을 활용하는 대표적인 예는 조직도 테이블

조직도 테이블

직원 이름(emp)	상관 이름(manager)	구내 번호(empTel)
나사장	NULL	0000
김재무	나사장	2222
김부장	김재무	2222-1
이부장	김재무	2222-2
우대리	이부장	2222-2-1
지사원	우대리	2222-2-2
이영업	나사장	1111
한과장	이영업	1111-1
최정보	나사장	3333
윤차장	최정보	3333-1
아주민	윤차장	3333-1-1

예시: 우대리 상관의 구내 번호 확인

```
SELECT A.emp AS '부하직원', B.emp AS '직속상관', B.empTel AS '직속상관연락처'
FROM empTBL A
INNER JOIN empTBL B
ON A.manager = B.emp
WHERE A.emp = '우대리';
```

결과

부하직원	직속상관	직속상관연락처
우대리	이부장	2222-2

UNION

- 두 쿼리의 결과를 행으로 합치는 연산자
- 기본 형태

```
SELECT 문장1 UNION
[ALL]
SELECT 문장2
```

예시 1) 단순 두 테이블을 모두 합치는 쿼리

```
SELECT stdName, addr FROM stdTBL
UNION [ALL]
SELECT clubName, roomNo FROM clubTBL;
```

- 중복된 열까지 모두 출력하려면 `UNION ALL`

IN, NOT IN

- 첫 번째 쿼리의 결과 중에서 두 번째 쿼리에 해당하는 것을 제외하고 출력

예시 1) cookDB의 사용자를 모두 출력하되 전화번호가 없는 사람을 제외

```
SELECT
    userName,
    CONCAT(mobile1, '-', mobile2) AS '전화번호'
FROM
    userTBL
WHERE
    userName NOT IN (SELECT userName FROM userTBL WHERE mobile1 IS NULL);
```

예시 2) 전화번호가 없는 사람만 조회

```
SELECT
    userName,
    CONCAT(mobile1, mobile2) AS '전화번호'
```

```
FROM
    userTBL
WHERE
    userName IN (SELECT userName FROM userTBL WHERE mobile1 IS NULL);
```

SQL 프로그래밍

- SQL에서도 다른 프로그래밍 언어와 비슷한 분기, 흐름 제어, 반복 등의 기능이 있음
- 프로시저(Procedure)로 복잡한 작업이나 반복 작업을 쉽게 구현 가능
- 기본 형태

```
DELIMITER $$
CREATE PROCEDURE 스토어드프로시저이름 ( )
BEGIN
    -- SQL 프로그래밍 코드
END $$
DELIMITER ;
CALL 스토어드프로시저이름 ( );
```

- `DELIMITER` : SQL 명령의 구분자를 변경하는 키워드. 프로시저 작성 시에는 `$` 또는 `$$` 로 변경하여 내부의 구문들을 처리 가능.
- `CREATE PROCEDURE` : 새 프로시저를 생성하는 명령.
- `CALL` : 생성된 프로시저를 실행하는 명령.

IF ... ELSE ... END IF

- 조건에 따라 분기하는 명령
- 명령을 실행한 결과, 한 문장 이상 처리해야 할 때는 `BEGIN ... END` 로 묶어야 함
- 기본 형태

```
IF <BOOL 표현식> THEN
    SQL문장들1 ...
ELSE
    SQL문장들2 ...
END IF;
```

예시 1) 변수 조건 분기

```
DROP PROCEDURE IF EXISTS ifProc;

DELIMITER $$
CREATE PROCEDURE ifProc()
BEGIN
    DECLARE var1 INT; -- 정수형 변수 선언
    SET var1 = 100; -- 변수에 값 대입

    IF var1 = 100 THEN -- var1이 100인지 비교
        SELECT '100입니다.';
    ELSE
        SELECT '100이 아닙니다.';
    END IF;
```

```
END $$
DELIMITER ;
CALL ifProc();
```

- `DECLARE` : 프로시저 내에서 변수를 선언하는 키워드.
- `SET` : 선언된 변수에 값을 대입.
- 조건 분기를 통해 변수의 값에 따라 출력 결과를 다르게 설정.

예시 2) 직원 입사일 확인

```
DROP PROCEDURE IF EXISTS ifProc2;
USE employees;

DELIMITER $$
CREATE PROCEDURE ifProc2()
BEGIN
    DECLARE hireDATE DATE; -- 입사일을 저장할 변수
    DECLARE curDATE DATE; -- 오늘 날짜를 저장할 변수
    DECLARE days INT; -- 날짜 차이를 저장할 변수

    SELECT hire_date INTO hireDate FROM employees.employees WHERE emp_no = 10001;

    SET curDATE = CURRENT_DATE(); -- 현재 날짜 설정
    SET days = DATEDIFF(curDATE, hireDATE); -- 입사일과 오늘의 차이 계산

    IF (days/365) >= 5 THEN -- 근무일수가 5년 이상인지 확인
        SELECT CONCAT('입사한지 ', days, '일이나 지났습니다. 축하합니다!') AS '메시지';
    ELSE
        SELECT '5년이 채 되지 않았습니다.' AS '메시지';
    END IF;
END $$
DELIMITER ;
CALL ifProc2();
```

- `DATEDIFF` : 두 날짜 간의 차이를 계산.
- `CURRENT_DATE()` : 현재 날짜 반환.
- 5년(365일 기준)을 초과했는지 조건을 설정하고 적합한 메시지를 출력.

CASE

- 다중 분기를 처리하는 구조

```
CASE
    WHEN 조건1 THEN 결과1
    WHEN 조건2 THEN 결과2
    ELSE 결과3
END;
```

예시 1) 점수에 따라 학점 분류

```
DROP PROCEDURE IF EXISTS caseProc;
DELIMITER $$
CREATE PROCEDURE caseProc()
```

```

BEGIN
    DECLARE point INT;          -- 점수 저장 변수
    DECLARE credit CHAR(1);    -- 학점을 저장할 변수
    SET point = 77;            -- 점수 초기화

    CASE
        WHEN point >= 90 THEN SET credit = 'A';
        WHEN point >= 80 THEN SET credit = 'B';
        WHEN point >= 70 THEN SET credit = 'C';
        WHEN point >= 60 THEN SET credit = 'D';
        ELSE SET credit = 'F';
    END CASE;

    SELECT CONCAT('취득점수==>', point), CONCAT('학점==>', credit);
END $$
DELIMITER ;
CALL caseProc();

```

예시 2) 회원 등급 구분하기

```

SELECT U.userID, U.userName, SUM(price * amount) AS 총구매액,
    CASE
        WHEN (SUM(price * amount) >= 1500) THEN '최우수고객'
        WHEN (SUM(price * amount) >= 1000) THEN '우수고객'
        WHEN (SUM(price * amount) >= 1) THEN '일반고객'
        ELSE '유령고객'
    END AS '고객등급'
FROM buyTBL B
RIGHT OUTER JOIN userTBL U
ON B.userID = U.userID
GROUP BY U.userID, U.userName
ORDER BY SUM(price * amount) DESC;

```

WHILE

- 다른 프로그래밍 언어와 동일한 방식으로 반복을 수행.
- <BOOL 식> 이 참인 동안 내부 SQL 명령문을 반복 실행.
- 기본 형태

```

WHILE <BOOL 식> DO
    SQL 명령문들 ...
END WHILE;

```

예시 1) 1부터 100까지 값을 모두 더하는 코드

```

DROP PROCEDURE IF EXISTS whileProc;
DELIMITER $$
CREATE PROCEDURE whileProc()
BEGIN
    DECLARE i INT; -- 1부터 100까지 증가할 변수
    DECLARE hap INT; -- 더한 값을 누적할 변수
    SET i = 1;
    SET hap = 0;

```

```

WHILE (i <= 100) DO
    SET hap = hap + i; -- hap에 현재 값을 더함
    SET i = i + 1; -- i를 1씩 증가
END WHILE;

SELECT hap; -- 결과 출력
END $$
DELIMITER ;
CALL whileProc();

```

결과: 5050

- hap은 현재까지 누적된 값을 저장.
- i는 현재 더해지는 값을 의미하며, 1씩 증가.

예시 2) 1부터 100 중 7의 배수를 합계에서 제외

```

DROP PROCEDURE IF EXISTS whileProc2;
DELIMITER $$
CREATE PROCEDURE whileProc2()
BEGIN
    DECLARE i INT; -- 1부터 100까지 증가할 변수
    DECLARE hap INT; -- 더한 값을 누적할 변수
    SET i = 1;
    SET hap = 0;

    myWhile: WHILE (i <= 100) DO
        -- 7의 배수인 경우 처리하지 않고 계속 진행
        IF (i % 7 = 0) THEN
            SET i = i + 1;
            ITERATE myWhile; -- WHILE로 다시 이동
        END IF;

        SET hap = hap + i;
        IF (hap > 1000) THEN
            LEAVE myWhile; -- WHILE 종료
        END IF;
        SET i = i + 1;
    END WHILE;

    SELECT hap;
END $$
DELIMITER ;
CALL whileProc2();

```

결과 : 1029

- ITERATE 문으로 7의 배수를 무시.
- LEAVE 문으로 누적 합이 1000을 초과하면 반복 종료.

PREPARE, EXECUTE (동적 SQL문)

- PREPARE 문은 SQL 문을 실행하지는 않고 따로 준비만 해놓으며, EXECUTE 문은 PREPARE문으로 준비 한 쿼리문을 실행
- 미리 쿼리문을 준비한 후 나중에 실행하는 것을 동적 SQL이라고 함

- 동적 SQL로 쿼리문을 실행한 후에는 DEALLOCATE PREPARE 문으로 준비 했던 문장을 해제하는 것이 좋음

예시)

```
USE cookDB;
PREPARE myQuery FROM 'SELECT * FROM userTBL WHERE userID = "NHS"';
EXECUTE myQuery;
DEALLOCATE PREPARE myQuery;
```

- PREPARE : SQL 문을 실행하지 않고 준비.
- EXECUTE : 준비된 SQL 문을 실행.
- DEALLOCATE PREPARE : 준비된 SQL 문을 해제.

예시 2) 동적 SQL과 현재 시간 삽입

```
USE cookDB;
DROP TABLE IF EXISTS myTable;
CREATE TABLE myTable (id INT AUTO_INCREMENT PRIMARY KEY, mDate DATETIME);

SET @curDATE = CURRENT_TIMESTAMP(); -- 현재 날짜와 시간 저장

PREPARE myQuery FROM 'INSERT INTO myTable VALUES(NULL, ?)';
EXECUTE myQuery USING @curDATE;
DEALLOCATE PREPARE myQuery;

SELECT * FROM myTable;
```

- 현재 시간을 동적으로 삽입하기 위해 SQL을 준비하고 실행
- ?로 동적 변수 사용. USING 구문으로 값을 바인딩